Agentic RAG for University Curriculum Navigation: An Empirical Study on Combining Semantic Retrieval with Structured Lookup Tools

Anh D. D. Hoang, Khang H. Tran, and Nam N. Trinh

***Abstract*—This paper presents the design, implementation, and evaluation of a Retrieval-Augmented Generation (RAG) chatbot that serves as an academic advisor for FPT University's Bachelor of IT in Artificial Intelligence (BIT_AI) program. The system combines semantic vector search over course syllabi with deterministic structured lookup tools for prerequisite chains, specialization pathways, and semester-level curriculum data. An agentic loop built on LangGraph enables the model to call multiple tools iteratively before synthesizing an answer. We evaluate the system on a 48-question golden test set across five categories using LLM-as-judge metrics. The RAG system achieves a correctness score of 0.625 compared to 0.240 for a baseline LLM-only system (160.4% relative improvement) and a faithfulness score of 0.865 versus 0.000 for the baseline. Per-category analysis shows the largest correctness gains in factual lookup (+0.550) and combo pathway navigation (+0.500) tasks.**

***Index Terms*—Retrieval-Augmented Generation, LangGraph, curriculum advising, agentic AI, LLM evaluation.**

I. Introduction

UNIVERSITY students frequently need guidance navigating complex curricula — understanding prerequisite chains, choosing specialization pathways, planning course loads, and determining which courses to retake. Traditional advising relies on human counselors who may be unavailable or unfamiliar with the latest curriculum changes. Large Language Models (LLMs) can provide conversational interfaces but hallucinate factual details such as credit counts, semester placements, and prerequisite relationships when operating without access to ground-truth data [1].

Retrieval-Augmented Generation (RAG) addresses this by grounding LLM responses in retrieved documents [2]. However, a standard RAG pipeline with a single vector search step has limitations for structured academic data: prerequisite chains require recursive resolution, semester-course mappings are tabular, and specialization pathways follow a fixed schema. These queries are better served by deterministic lookups than by semantic similarity.

This work presents an **Agentic RAG** system that combines semantic vector search with structured JSON lookup tools, orchestrated by a LangGraph [3] state machine. The agent decides which tools to invoke, can call multiple tools in sequence, and includes a document relevance grading step with query rewriting. We evaluate the system against a no-retrieval baseline on a curated 48-question test set covering the BIT_AI K20-K21 curriculum at FPT University.

The contributions of this paper are:

1) A hybrid retrieval architecture combining vector search with four structured lookup tools for academic curriculum data.
2) A LangGraph-based agentic loop with document grading and multi-tool iteration.
3) An empirical evaluation comparing RAG versus LLM-only on faithfulness, relevancy, and correctness across five question categories.

II. Related Work

RAG was introduced by Lewis et al. [2] to mitigate LLM hallucination by conditioning generation on retrieved passages. Subsequent work explored advanced retrieval strategies including re-ranking [6], query decomposition, and hybrid search. Gao et al. [1] provide a comprehensive survey of RAG techniques and their applications.

Agentic RAG. ReAct [4] introduced the paradigm of interleaving reasoning and tool use in LLMs. LangGraph [3] operationalizes this pattern with a state machine abstraction, enabling conditional routing, tool invocation, and iterative refinement loops. Our work adopts this architecture for multi-tool academic advising.

LLM-as-Judge Evaluation. Zheng et al. [5] demonstrated that LLMs can serve as reliable evaluators of open-ended generation quality, correlating well with human judgments. We use GPT-4o-mini as a judge to score answer relevancy, faithfulness, and correctness.

Educational AI. Prior work on AI tutoring systems [7] has focused on content delivery and assessment. Our system targets curriculum navigation specifically — a planning-oriented task that requires combining information across multiple structured data sources.

III. System Architecture

A. Data Corpus

The raw corpus consists of 9 markdown files covering the BIT_AI K20-K21 curriculum:

- **Syllabi**: Individual course files containing descriptions, learning outcomes, session schedules, assessment structures, and materials.

The authors are with Information Technology Specialization, FPT University, Hanoi, Vietnam.

Manuscript prepared March 2026.

- **Curriculum**: A master document listing all 47 courses across 10 semesters (semesters 0–9) with credit counts and prerequisites.
- **Pathways**: Specialization combo definitions with elective course groupings organized by topic.

Each file is classified by regex matching on its H1 header into one of three entity types: `syllabus`, `curriculum`, or `pathway`.

B. Ingestion Pipeline

The ingestion pipeline transforms raw markdown into two parallel representations:

Vector index. Documents are split using a `MarkdownHeaderTextSplitter` at H2 section boundaries, then oversized chunks are sub-split with a `RecursiveCharacterTextSplitter` (chunk size 500 characters, overlap 50). A minimum chunk length filter of 50 characters removes fragments from table-row splits, reducing the total from 8,446 to 6,256 chunks. Chunks are embedded with OpenAI `text-embedding-3-small` (1,536 dimensions) and indexed in Pinecone Serverless (cosine similarity).

Structured tables. Six JSON lookup tables are extracted deterministically from the parsed documents:

1) `course_index`: 47 entries mapping course codes to name, credits, semester, prerequisite, and description.
2) `prerequisites`: 23 entries mapping course codes to their prerequisite strings.
3) `curriculum_map`: 10 semesters, each with a list of courses.
4) `combo_map`: 5 specialization pathways with course groupings.
5) `programme_outcomes`: 5 program-level outcomes.
6) `program_learning_outcomes`: 13 PLOs.

C. Tool Definitions

Four LangChain tools are defined, each backed by either the vector index or the JSON tables:

1) **vector_search**(q, c, t): Semantic search over Pinecone. Accepts optional `course_code` c for metadata filtering and `entity_type` $t \in \{\text{syllabus}, \text{curriculum}, \text{pathway}\}$. Returns top-$k$ (k=5) chunks with source metadata.
2) **prerequisite_lookup**(c, d): Deterministic prerequisite resolver. Direction d=forward returns the full recursive prerequisite chain; d=reverse finds courses that list c as a prerequisite.
3) **combo_navigator**(id, t): Browse specialization pathways by ID or topic name search. Returns course listings with active/inactive status.
4) **curriculum_browser**(s, c): Browse the study plan by semester s or look up a specific course c. Returns course names, credits, and prerequisites.

All tools include a fuzzy course code resolver that handles partial input (e.g., "AIL" → "AIL303m") via prefix matching against the course index.

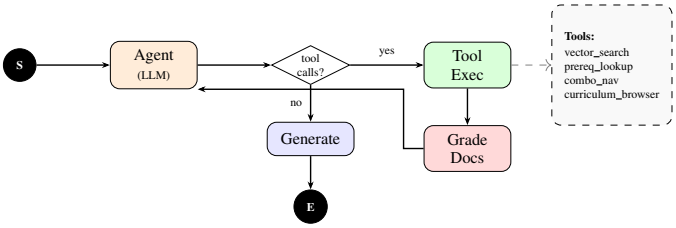

Fig. 1. LangGraph agent state machine. The agent–tools–grade loop repeats up to 6 iterations. Structured tools bypass grading; only `vector_search` results are graded for relevance.

D. LangGraph Agent

The agent is implemented as a LangGraph `StateGraph` with four nodes and conditional routing:

Algorithm 1 Agent graph execution.

```
s ← INITSTATE(question, history, profile)
repeat
  response ← AGENTNODE(s)          // LLM with tools
  if response has tool calls then
    results ← TOOLNODE(response.tool_calls)
    grade ← GRADEDOCUMENTS(results)
    if grade = not_relevant and retries < 1 then
      Rewrite query, increment retry
    end if
    Increment tool_call_count
  end if
until no tool calls or tool_call_count ≥ 6
answer ← GENERATENODE(current_turn_messages)
return answer
```

Fig. 1 illustrates the graph structure visually.

Structured tools (prerequisite_lookup, combo_navigator, curriculum_browser) skip the grading step since they return deterministic data. Only `vector_search` results are graded for relevance by an LLM judge, with one query rewrite attempt if irrelevant.

The GENERATENODE receives only the current turn's messages (extracted by walking backward to the last `HumanMessage`) to prevent prior conversation turns from bleeding into the response.

E. Student Personalization

The system supports student profiles containing passed courses with grades, failed courses, and current semester. Profile data is injected into the system prompt as structured context. The agent cross-references this with tool results to provide personalized advice: flagging blocked courses due to failed prerequisites, suggesting retake candidates, and building acceleration plans respecting FPT's rule of at most 2 extra courses per semester from the next semester.

IV. Experimental Setup

A. Test Set

We curated a golden test set of 48 question–answer pairs across five categories:

TABLE I
GOLDEN TEST SET COMPOSITION

Category	Count	Example Question Type
Factual Lookup	10	Course credits, semester placement
Prerequisite Reasoning	10	Dependency chains, reverse lookup
Combo Pathway	8	Specialization courses, electives
Cross Document	10	Multi-source synthesis
Curriculum Planning	10	Acceleration, semester planning
Total	**48**	

TABLE II
OVERALL EVALUATION METRICS

Metric	RAG	Baseline	Δ
Relevancy	0.958	0.948	+0.010
Faithfulness	0.865	0.000	+0.865
Correctness	0.625	0.240	+0.385

Each entry includes a reference answer, expected tool, and expected sources for evaluation grounding.

B. Metrics

We use LLM-as-judge evaluation [5] with GPT-4o-mini scoring three metrics on a $[0, 1]$ scale:

- **Answer Relevancy**: Does the answer address the question asked?
- **Faithfulness**: Is the answer grounded in the retrieved context (not hallucinated)?
- **Correctness**: Does the answer match the reference answer?

C. Systems Under Evaluation

RAG system: The full agentic pipeline with all four tools, document grading, and query rewriting. Uses GPT-4o-mini (`temperature=0`) for both agent reasoning and final generation.

Baseline: GPT-4o-mini with a minimal system prompt ("You are an academic advisor for FPT University's BIT_AI program") and no access to tools or retrieval. This isolates the improvement attributable to RAG.

Both systems were evaluated on all 48 questions. Tool calls, generated answers, and judge scores with reasoning were recorded for each question.

V. RESULTS

A. Overall Performance

Table II summarizes the aggregate metrics across all 48 questions.

The RAG system achieves a correctness score 160.4% higher than the baseline in relative terms (+0.385 absolute). Faithfulness improves from 0.000 to 0.865 — the baseline receives a score of zero because it has no context to be faithful to, while the RAG system grounds its answers in tool results. Relevancy is comparable between the two systems (0.958 vs. 0.948), indicating that both understand the questions well; the difference lies in answer accuracy.

Fig. 2 visualizes this comparison.

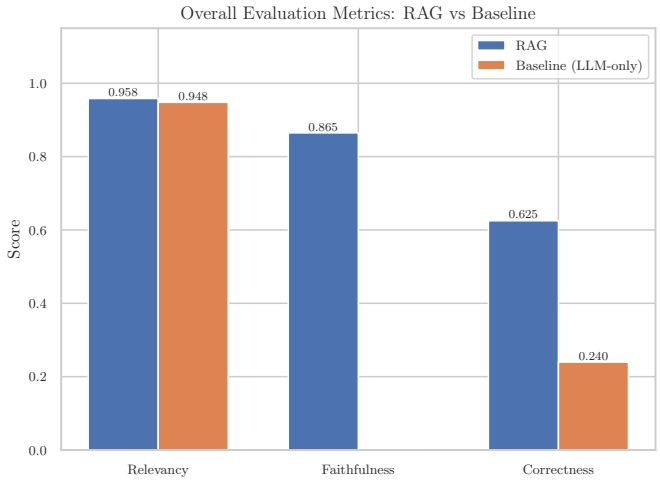

Fig. 2. Overall evaluation metrics comparing the RAG system with the LLM-only baseline across all 48 test questions.

TABLE III
CORRECTNESS BY QUESTION CATEGORY

Category	RAG	Baseline	Δ
Factual Lookup	0.700	0.150	+0.550
Combo Pathway	0.688	0.188	+0.500
Prerequisite Reasoning	0.600	0.150	+0.450
Curriculum Planning	0.600	0.250	+0.350
Cross Document	0.550	0.450	+0.100

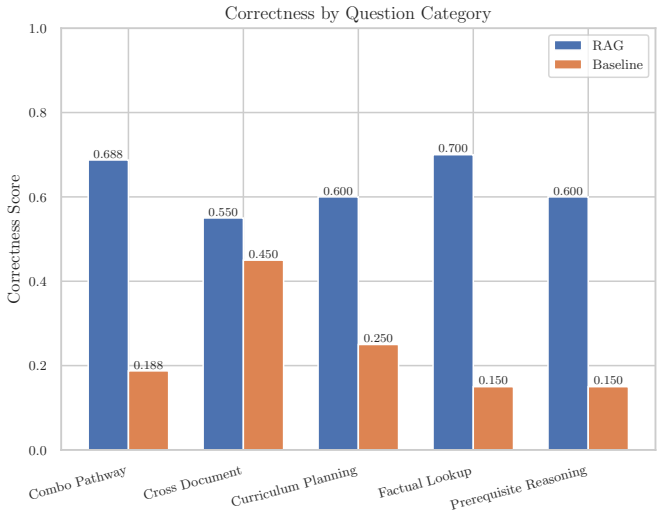

Fig. 3. Per-category correctness scores. The RAG system shows the largest improvements in Factual Lookup and Combo Pathway categories.

B. Per-Category Analysis

Table III breaks down correctness scores by question category.

The largest improvements appear in **Factual Lookup** (+0.550) and **Combo Pathway** (+0.500), where the baseline frequently hallucinates incorrect credit counts, semester placements, and course codes. The smallest improvement is in **Cross Document** (+0.100), where both systems achieve

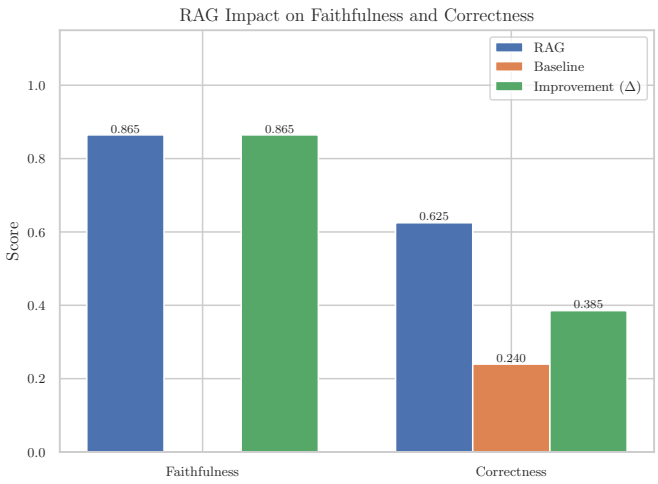

Fig. 4. RAG impact on faithfulness and correctness. The baseline achieves zero faithfulness because it has no retrieved context to be faithful to.

moderate scores. This category requires synthesizing information across multiple documents, a task where vector search retrieval is less precise because the query must match content scattered across different files.

C. Faithfulness and Correctness Impact

Fig. 4 isolates the impact of retrieval on faithfulness and correctness.

The faithfulness gap (0.865 vs. 0.000) demonstrates that tool access is necessary for grounded answers. The baseline LLM generates plausible-sounding but unsupported claims — for example, stating that a course has 3 credits when it actually has 6, or placing a course in semester 6 when it belongs to semester 4.

D. Score Distribution

Fig. 5 shows the distribution of per-question correctness scores for both systems.

The RAG system produces 19 fully correct answers (score = 1.0) compared to 5 for the baseline. Conversely, the RAG system has 9 failures (score = 0.0) versus 27 for the baseline. The baseline is concentrated at 0.0 and 0.5, while the RAG system shifts the distribution rightward toward higher correctness.

E. Tool Usage Patterns

Fig. 6 shows the frequency of each tool's invocation across the 48 evaluation questions.

`vector_search` is the most frequently invoked tool, used for content questions, assessment details, and learning outcomes. `prerequisite_lookup` and `curriculum_browser` are used for structured queries about dependencies and semester plans. `combo_navigator` is invoked less frequently, corresponding to the smaller number of combo/pathway questions in the test set.

F. Per-Question Analysis

Fig. 7 shows the per-question correctness delta (RAG minus baseline) across all 48 questions.

The majority of questions show zero or positive delta, with 29 questions showing improvement from RAG and only 2 questions where the baseline outperforms. The 17 remaining questions have equal scores for both systems.

G. Failure Analysis

We identify three failure modes in the RAG system:

Credit value confusion (fact_01, fact_04, fact_09): The system occasionally returns incorrect credit values when the `vector_search` tool retrieves chunks that do not contain the credit field, and the `curriculum_browser` is not invoked as a fallback.

Incomplete prerequisite chains (prereq_03, prereq_06): Deep prerequisite hierarchies spanning 3+ levels are sometimes partially resolved, with intermediate requirements omitted.

Cross-document synthesis (content_03, content_09): Questions requiring information from multiple unrelated documents (e.g., comparing assessment methods of two courses) perform worst, as the vector search must be invoked multiple times and the retrieved chunks may not cover all relevant aspects.

The baseline system exhibits systematic failures: inventing nonexistent course codes, assigning incorrect semester numbers (e.g., semester 6 instead of 4), and reporting inaccurate credit totals (40 subjects instead of 47).

VI. Discussion

A. Why Structured Tools Over Graph RAG

An initial design proposal suggested Graph RAG for this project, given the relational nature of curriculum data (prerequisite dependencies, combo groupings, semester ordering). We chose standard RAG with structured JSON lookup tools for three reasons:

1) **Small corpus**: With only 9 source documents and 47 courses, the overhead of building and maintaining a knowledge graph is not justified.
2) **Explicit relationships**: Prerequisites are enumerated in tables, not implied by context. A JSON lookup with recursive chain resolution handles them deterministically, avoiding the probabilistic nature of graph traversal over embeddings.
3) **Evaluation clarity**: Deterministic tools produce verifiable outputs, making correctness evaluation straightforward.

The evaluation results support this decision: structured tools (`prerequisite_lookup`, `curriculum_browser`, `combo_navigator`) achieve higher per-question correctness than `vector_search` on their respective question categories.

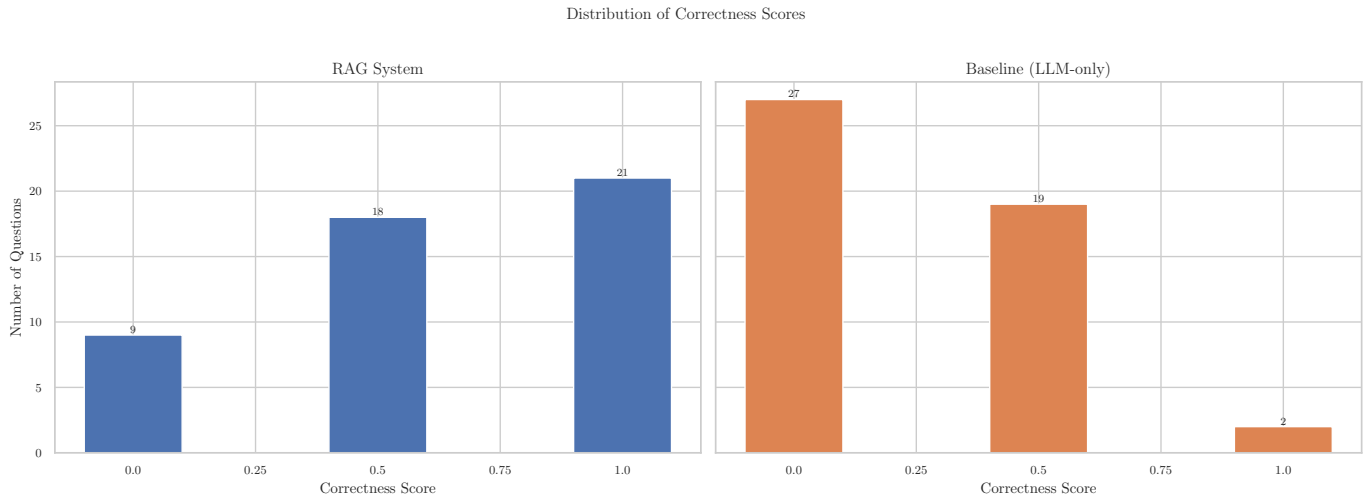

Fig. 5. Distribution of correctness scores across all 48 questions. The RAG system produces more fully correct (1.0) answers and fewer failures (0.0) than the baseline.

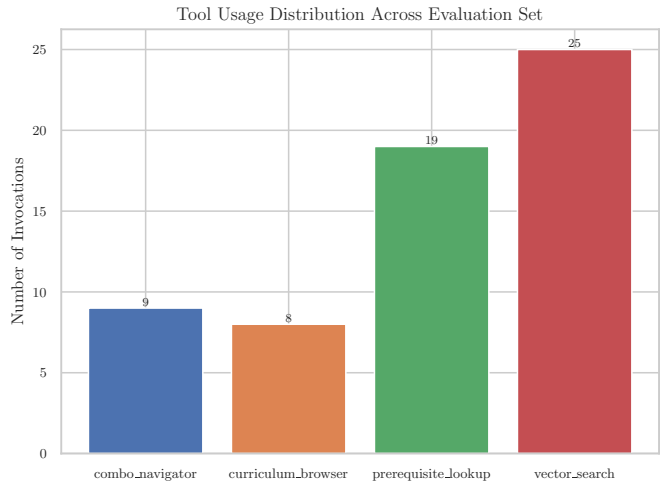

Fig. 6. Tool usage distribution across the 48 evaluation questions.

B. Multi-Tool Iteration

A key architectural contribution is the multi-tool loop. The original single-pass design (agent → tool → generate) could only invoke one tool per question. Complex queries such as "Can I accelerate to Deep Learning?" require both `prerequisite_lookup` (to resolve the dependency chain) and `curriculum_browser` (to identify semester placements). The looping design allows the agent to gather all necessary data before synthesis.

C. Document Grading and Query Rewriting

The grading step is applied only to `vector_search` results, as structured tools return deterministic data that is always relevant. When vector search results are judged irrelevant, the query is rewritten once. We observed that the rewriter sometimes expands partial course codes incorrectly (e.g., "MAE" → "Master of Arts in Education"), which we mitigated by adding explicit instructions to preserve course code prefixes.

D. Limitations

The system has several limitations:

- The evaluation uses LLM-as-judge rather than human evaluation. While prior work shows high correlation between LLM and human judgments [5], bias from the same model family (GPT-4o-mini) evaluating its own outputs cannot be ruled out.
- The test set of 48 questions, while covering five categories, is relatively small. A larger evaluation set would provide more statistical confidence.
- The system covers only the K20-K21 curriculum version. Extending to multiple curriculum versions would require versioned structured tables.
- Cross-document questions remain challenging, suggesting that the single-query vector search approach is insufficient for multi-source synthesis tasks.

VII. Conclusion

We presented an agentic RAG system for university curriculum navigation that combines semantic vector search with deterministic structured lookup tools. The LangGraph-based agent can iteratively invoke multiple tools, grade document relevance, and rewrite queries before synthesizing a final answer. Evaluated on a 48-question test set, the system achieves 0.625 correctness compared to 0.240 for an LLM-only baseline (+160.4% relative), with faithfulness improving from 0.000 to 0.865. The largest gains appear in factual lookup and specialization pathway questions, where deterministic tools provide precise answers that the baseline model would otherwise hallucinate.

Future work includes extending the evaluation with human judges, supporting multiple curriculum versions, improving cross-document retrieval through multi-hop strategies, and deploying the system for real student use with feedback collection.

Fig. 7. Per-question correctness improvement (Δ = RAG − Baseline). Positive values (green) indicate RAG outperformance; negative values (red) indicate baseline outperformance.

Acknowledgments

This work was conducted as part of the BIT_AI program at FPT University.

References

[1] Y. Gao, Y. Xiong, X. Xu, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.

[2] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.

[3] LangChain, "LangGraph: Build stateful, multi-actor applications with LLMs," 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[4] S. Yao, J. Zhao, D. Yu, et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.

[5] L. Zheng, W.-L. Chiang, Y. Sheng, et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Proc. NeurIPS*, 2023.

[6] N. Nogueira, W. Jiang, R. Pradeep, and J. Lin, "Document Ranking with a Pretrained Sequence-to-Sequence Model," in *Proc. Findings of EMNLP*, 2020, pp. 708–718.

[7] K. R. Koedinger and A. Corbett, "Cognitive Tutors: Technology Bringing Learning Sciences to the Classroom," in *The Cambridge Handbook of the Learning Sciences*, 2nd ed., R. K. Sawyer, Ed. Cambridge Univ. Press, 2014, pp. 61–78.